

Ultimative Guide: Python Listen (List)

Was ist eine Liste?

Eine Liste ist eine geordnete, veränderbare Sammlung von Elementen in Python.

python

```
# Eine einfache Liste erstellen
meine_liste = [1, 2, 3, 4, 5]
namen = ["Anna", "Ben", "Clara"]
gemischt = [1, "Hallo", 3.14, True]
```

Listen erstellen

python

```
# Leere Liste
leere_liste = []
leere_liste2 = list()

# Liste mit Elementen
zahlen = [1, 2, 3, 4, 5]
woerter = ["Apfel", "Banane", "Kirsche"]

# Liste mit range()
zahlen_bis_10 = list(range(1, 11)) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Liste aus String
buchstaben = list("Python") # ['P', 'y', 't', 'h', 'o', 'n']
```

Auf Elemente zugreifen

python

```
fruits = ["Apfel", "Banane", "Kirsche", "Datteln", "Erdbeere"]

# Einzelne Elemente
print(fruits[0]) # "Apfel" - erstes Element
print(fruits[2]) # "Kirsche" - drittes Element
print(fruits[-1]) # "Erdbeere" - letztes Element
print(fruits[-2]) # "Datteln" - vorletztes Element

# Slicing [start:stop:step]
print(fruits[1:3]) # ["Banane", "Kirsche"]
print(fruits[:3]) # ["Apfel", "Banane", "Kirsche"]
print(fruits[2:]) # ["Kirsche", "Datteln", "Erdbeere"]
print(fruits[::2]) # ["Apfel", "Kirsche", "Erdbeere"] - jedes 2. Element
print(fruits[::-1]) # Umgekehrte Reihenfolge
```

Listen modifizieren

python

```
zahlen = [1, 2, 3]

# Elemente hinzufügen
zahlen.append(4) # [1, 2, 3, 4]
zahlen.insert(1, 1.5) # [1, 1.5, 2, 3, 4] - bei Index 1 einfügen
zahlen.extend([5, 6]) # [1, 1.5, 2, 3, 4, 5, 6]

# Elemente entfernen
zahlen.remove(1.5) # [1, 2, 3, 4, 5, 6] - Wert entfernen
entfernt = zahlen.pop(2) # [1, 2, 4, 5, 6] - bei Index 2 entfernen
zahlen.clear() # [] - Liste leeren

# Elemente ändern
zahlen = [1, 2, 3]
zahlen[1] = 20 # [1, 20, 3]
zahlen[0:2] = [10, 20] # [10, 20, 3]
```

Listen Operationen

python

```
liste1 = [1, 2, 3]
liste2 = [4, 5, 6]

# Verknüpfung
kombiniert = liste1 + liste2 # [1, 2, 3, 4, 5, 6]

# Wiederholung
vervielfacht = liste1 * 3 # [1, 2, 3, 1, 2, 3, 1, 2, 3]

# Länge
laenge = len(liste1) # 3

# Prüfen ob Element enthalten
ist_enthalten = 2 in liste1 # True
```

Listen Methoden

python

```
zahlen = [3, 1, 4, 1, 5, 9, 2]

# Sortieren
zahlen.sort() # [1, 1, 2, 3, 4, 5, 9]
zahlen.sort(reverse=True) # [9, 5, 4, 3, 2, 1, 1]

# Umkehren
zahlen.reverse() # [1, 1, 2, 3, 4, 5, 9] → [9, 5, 4, 3, 2, 1, 1]

# Index finden
position = zahlen.index(4) # 2 (erstes Vorkommen von 4)

# Zählen
anzahl = zahlen.count(1) # 2 (wie oft kommt 1 vor)

# Kopieren
kopie = zahlen.copy() # Erstellt eine flache Kopie
kopie2 = zahlen[:] # Alternative für Kopie
```

Listen durchlaufen

python

```
fruits = ["Apfel", "Banane", "Kirsche"]

# Mit for-Schleife
for fruit in fruits:
    print(fruit)

# Mit Index
for i in range(len(fruits)):
    print(f"{i}: {fruits[i]}")

# Mit enumerate (Index und Wert)
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Ausgabe:
# 0: Apfel
# 1: Banane
# 2: Kirsche
```

List Comprehension

Elegante Art, Listen zu erstellen und zu transformieren.

python

```
# Einfache List Comprehension
quadrat = [x**2 for x in range(5)]      # [0, 1, 4, 9, 16]

# Mit Bedingung
gerade_zahlen = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]

# Strings transformieren
woerter = ["hallo", "welt", "python"]
gross = [wort.upper() for wort in woerter]  # ["HALLO", "WELT", "PYTHON"]

# Verschachtelte Schleifen
kombinationen = [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
# [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

Mehrdimensionale Listen

python

```
# 2D-Liste (Matrix)
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Zugriff
print(matrix[0][1])  # 2 (Zeile 0, Spalte 1)
print(matrix[2][0])  # 7 (Zeile 2, Spalte 0)

# Durch 2D-Liste iterieren
for zeile in matrix:
    for element in zeile:
        print(element, end=" ")
    print()  # Neue Zeile

# Ausgabe:
# 1 2 3
# 4 5 6
# 7 8 9
```

Nützliche Funktionen mit Listen

python

```
zahlen = [5, 2, 8, 1, 9]

# Summe
summe = sum(zahlen)      # 25

# Maximum/Minimum
maximum = max(zahlen)    # 9
minimum = min(zahlen)    # 1

# Sortierte Kopie
sortiert = sorted(zahlen)  # [1, 2, 5, 8, 9] - originale Liste unverändert

# Any/All
hat_gerade = any(x % 2 == 0 for x in zahlen)  # True (mind. eine gerade Zahl)
alle_gerade = all(x % 2 == 0 for x in zahlen)  # False (nicht alle gerade)
```

Listen und Strings

python

```
# String zu Liste
text = "Hallo Welt"
woerter = text.split()      # ["Hallo", "Welt"]
buchstaben = list(text)     # ["H", "a", "l", "l", "o", " ", "W", "e", "l", "t"]

# Liste zu String
liste = ["Python", "ist", "toll"]
text = " ".join(liste)      # "Python ist toll"
pfad = "/".join(["home", "user", "documents"])  # "home/user/documents"
```

Praktische Beispiele

python

```
# Einkaufsliste
einkaufsliste = ["Brot", "Milch", "Eier"]
einkaufsliste.append("Butter")
einkaufsliste.remove("Milch")
print("Ich muss kaufen:", ", ".join(einkaufsliste))

# Zahlen verarbeiten
zahlen = [x for x in range(1, 101) if x % 7 == 0] # Alle Zahlen von 1-100, die durch 7 teilbar sind

# Notendurchschnitt berechnen
noten = [2.3, 1.7, 3.0, 2.0, 1.3]
durchschnitt = sum(noten) / len(noten) # 2.06

# Einfache To-Do Liste
todo_liste = []
todo_liste.append("Python lernen")
todo_liste.append("Einkaufen gehen")
todo_liste.append("Sport machen")

erledigt = todo_liste.pop(0) # Erstes Element entfernen und zurückgeben
print(f"Erledigt: {erledigt}")
print(f"Noch zu tun: {todo_liste}")
```

Wichtige Hinweise

python

```
# Referenzen vs. Kopien
liste_a = [1, 2, 3]
liste_b = liste_a # Referenz - beide zeigen auf dasselbe Objekt
liste_b.append(4)
print(liste_a) # [1, 2, 3, 4] - liste_a wurde auch verändert!

liste_c = liste_a.copy() # Echte Kopie - unabhängiges Objekt
liste_c.append(5)
print(liste_a) # [1, 2, 3, 4] - unverändert
```

Dieser Guide deckt die wichtigsten Konzepte von Python-Listen ab. Mit diesem Wissen kannst du Listen effektiv in deinen Programmen einsetzen!